



CMPT 295

Unit - Data Representation

Lecture 3 – Representing (encoding) integral numbers in memory
- unsigned and signed

Last Lecture

- **von Neumann** architecture - architecture of most computers
 - Its components: CPU, memory, input and output, bus
 - One of its characteristics: Data and instructions (code/programs) both stored in memory
- A look at **memory**: defined **byte-addressable** memory, compressed view of memory
- A look at **memory content**
 - Why we abstract the content of a memory cell using the two values of the binary numeral system -> '0' and '1'
 - Algorithm for converting binary to hexadecimal (hex)
 1. Partition bit vector into groups of 4 bits, starting from right, i.e., least significant **byte** (LSB)
 - If most significant **byte** (MSB) does not have 8 bits, pad it: add 0's to its left
 2. Translate each group of 4 bits into its corresponding hex value
 - **little endian** versus **big endian**: Order in which bytes are stored in memory
- **Bit manipulation** – regardless of what a bit pattern (vector) represents/means
 - **Boolean algebra**: **bitwise operations** => **AND** (&), **OR** (|), **XOR** (^), **NOT** (~)
 - **Shift operations**: left shift $x \ll y$, logical and arithmetic right shift $x \gg y$
 - **Right logical shift**: Fill x with y 0's on the left of x
 - **Right arithmetic shift**: Fill x with y copies of x 's sign bit on the left of x
 - **Sign bit** is most significant **bit** (MSb) before shifting occurred

reviewed in Lab 0

Determined by a microprocessor. Describes how a microprocessor orders the bytes when it reads them from and writes them to memory.

NOTE:

C logical operators and C bitwise (bit-level) operators behave differently!
Watch out for && versus &, || versus |, ...

Today's Menu

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

Demo

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

- Bits representing (encoding) real numbers

- IEEE floating point representation

- Floating point in C – casting, rounding, addition, ...

What could these 32 bits represent?

- What kind of information could they encode?

0 1 1 0 0 0 1 0 0 1 1 0 1 0 0 1 0 1 1 1 0 1 0 0 0 1 1 1 0 0 1 ₂

Answer:

What kind of information do **bit patterns** represent?

Encoding Scheme

Bit pattern
01100010 01101001
01110100 01110011₂



- ASCII character
- Unsigned integral
- Two's complement (signed) integral
- Floating point (IEEE 754)
- Memory Address
- Assembly language
- RGB
- MP3
- ...



- Letters and symbols
- Positive numbers
- Negative numbers
- Real numbers
- C pointers
- Machine instructions
- Colour
- Audio/Sound
- ...

Definition: An **encoding scheme** is an interpretation of a bit pattern

Bottom line: Which encoding scheme is used to interpret a bit pattern depends on the application currently executing (the **context**)

Lab 1 - Representing integral numbers in memory - unsigned and signed

Produce the same result even though the encoding scheme equations are not the same.

- Lab 1 - **Review** of encoding schemes – giving meaning to bits in memory
- In this lab, we shall practice the following encoding schemes:
 - **Bits** $\Leftrightarrow$ **Unsigned integral** (positive numbers)
 - **Bits** $\Leftrightarrow$ **Signed integral** (positive numbers)
 - **Bits** $\Leftrightarrow$ **Signed integral** (negative numbers)
- Posted on our course web site
- Nothing to hand in
- Worth 0 marks
- Invitation to come to my office hours if you have questions

Examples of “Show your work”

Lab 1 – Quick Check

Let's do a quick check of the material covered in Lab 1 before we continue ...

1. Convert $x = 11110100_2$ into a signed integral ($\text{B2T}(x)$) if $w = 8$:

Examples of "Show your work"

Lab 1 – Quick Check

2. Convert $x = -12$ into a binary number ($T2B(x)$) if $w = 8$:

Method 1 If $x < 0$,
 $T2B(X) = (\sim(U2B(|X|))) + 1$

If $x = -12$

1. $|x| \Rightarrow |-12| =$

2. $U2B(12) \Rightarrow$

3. $\sim(00001100_2) \Rightarrow$

4. $(11110011_2) + 1 \Rightarrow$

Binary addition:

$$\begin{array}{r} 11110011 \\ + 00000001 \\ \hline \end{array}$$

Method 2 If $x < 0$,
 $T2B(X) = U2B(T2U(X)) = U2B(X + 2^w)$

If $x = -12$

1. $x + 2^w \Rightarrow -12 + 256 = 244$

2. $U2B(244) \Rightarrow 11110100_2$

Using subtraction:

$244 - 128 = 116 \rightarrow 1 \times 2^7$

$116 - 64 = 52 \rightarrow 1 \times 2^6$

$52 - 32 = 20 \rightarrow 1 \times 2^5$

$20 - 16 = 4 \rightarrow 1 \times 2^4$

$4 - 8 \rightarrow \text{nop!} \rightarrow 0 \times 2^3$

$4 - 4 \rightarrow 0 \rightarrow 1 \times 2^2$

$0 - 2 \rightarrow \text{nop!} \rightarrow 0 \times 2^1$

$0 - 1 \rightarrow \text{nop!} \rightarrow 0 \times 2^0$

Always check your answer! How? Reverse the conversion process: $128 + 64 + 32 + 16 + 4 = 244$ then $244 - 256 = -12$ ☺

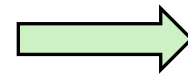
Let's now continue ...

Properties of unsigned & signed conversions

$w = 4$

X	$B2U(X)$	$B2T(X)$
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

➤ Equivalence



➤ Both encoding schemes ($B2U$ and $B2T$) produce the **same bit patterns for nonnegative values**

➤ Uniqueness

➤ Every bit pattern produced by these encoding schemes ($B2U$ and $B2T$) represents a **unique** (and **exact**) integer value

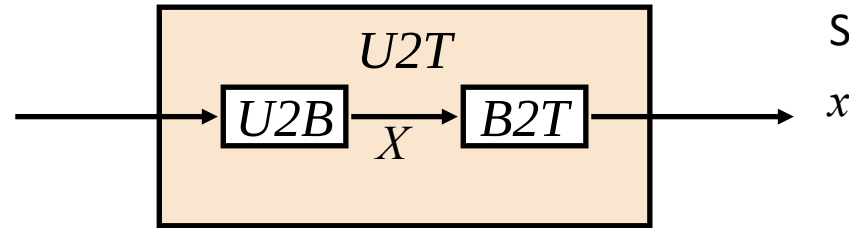
➤ Each representable integer has **unique bit pattern**

Converting unsigned $\leftrightarrow$ signed – Method 1 of same size (same data type)

$w = 8$

If $ux = 129_{10}$

Unsigned
 ux



Signed (Two's Complement)
 x

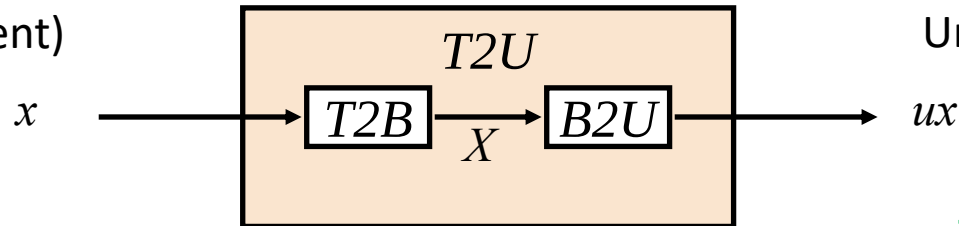
Maintain Same Bit Pattern

then $x =$

$w = 4$

Signed (Two's Complement)
 x

If $x = -5_{10}$



Unsigned
 ux

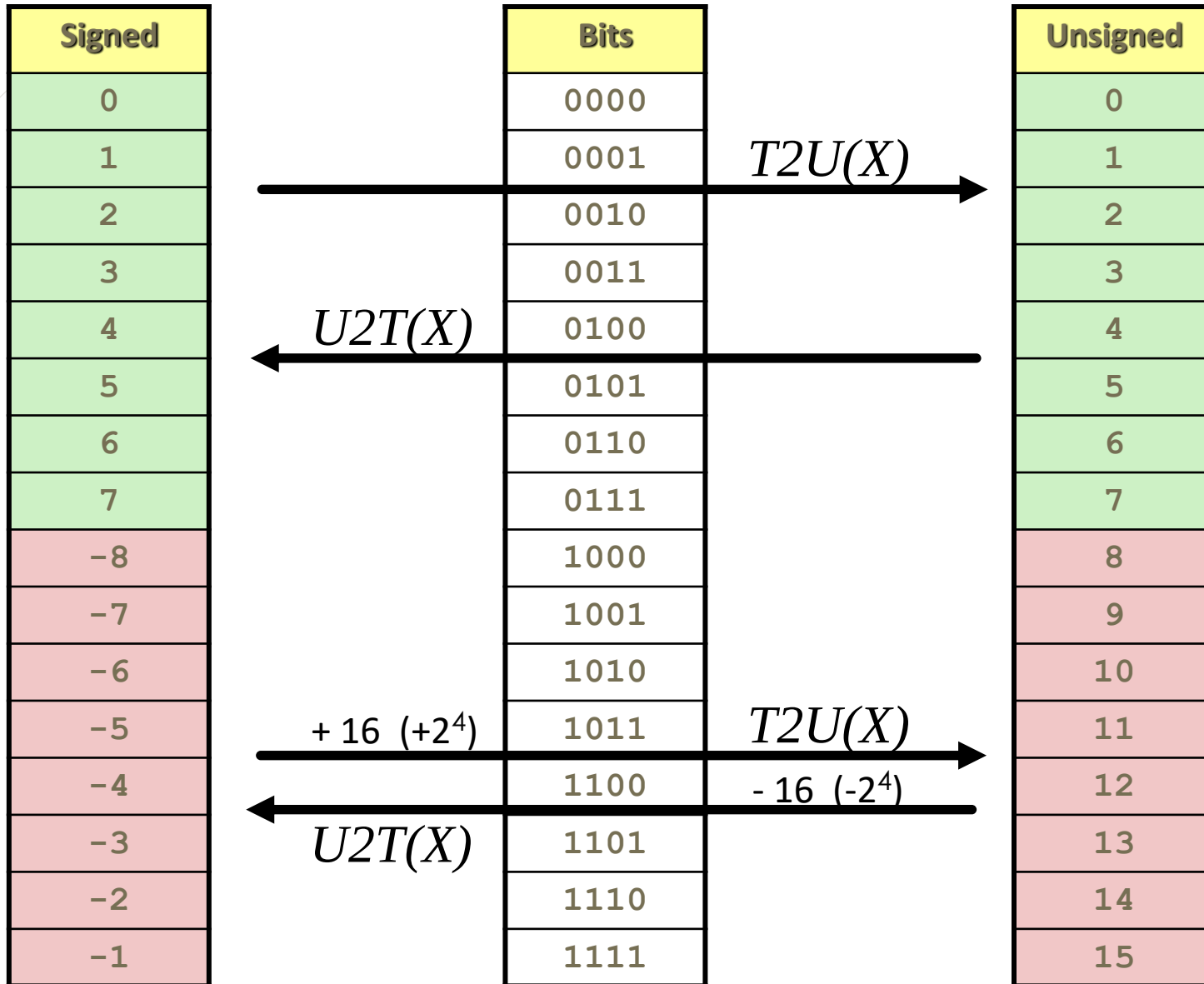
Maintain Same Bit Pattern

then $ux =$

- **Conclusion** - Converting between unsigned and signed numbers: Both have **same bit pattern**, however, this bit pattern may be **interpreted differently**, i.e., producing a different integral value

Converting unsigned $\leftrightarrow$ signed – Method 2

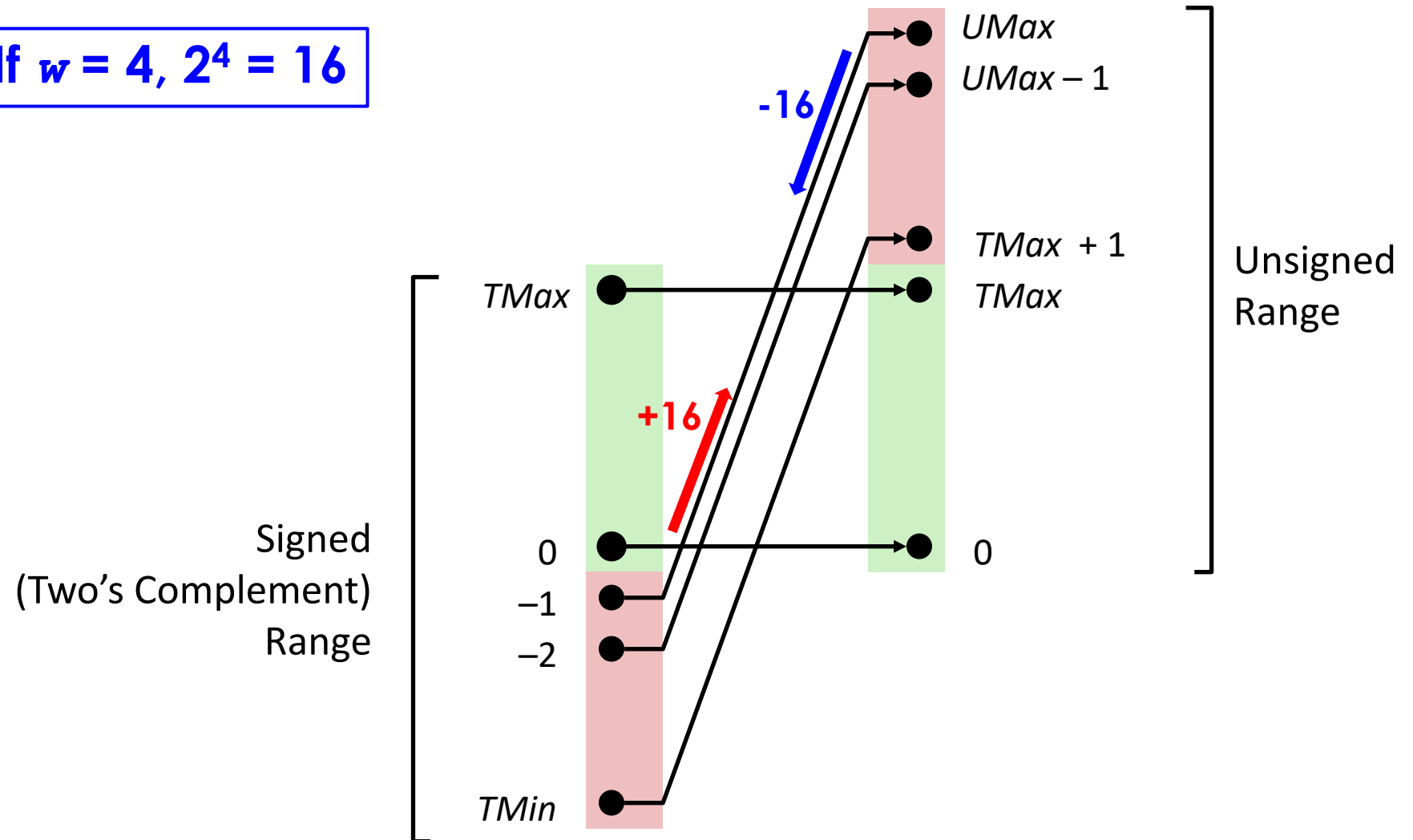
$w = 4$



Step 1 of
Method 2
on Slide 10

Visualizing the relationship between signed & unsigned

If $w = 4$, $2^4 = 16$



Let's now consider **C** on our *target machine*

- Let's now investigate the effect of all this on our *target machine* when we are executing a **C** program
- As a preliminary exercise, complete the following:

- char => _____ bits -> _____ byte
- short => _____ bits -> _____ bytes
- int => _____ bits -> _____ bytes
- long => _____ bits -> _____ bytes
- float => _____ bits -> _____ bytes
- double => _____ bits -> _____ bytes
- pointer=> _____ bits -> _____ bytes
(e.g. char *)

Homework

Range of possible values?

➤ If the context is **C** on our *target machine*, what is the range of possible values for the following **C** data types:

➤ unsigned char? []

➤ signed short? []

➤ unsigned int? []

➤ unsigned long? []

➤ signed int? []

➤ signed char? []

➤ unsigned short? []

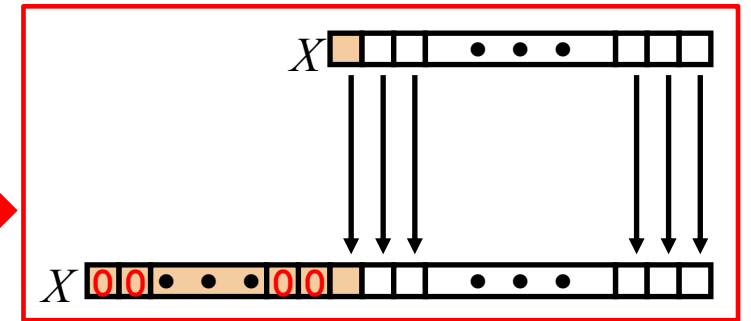
➤ signed long? []

Converting unsigned (or signed) of different sizes (i.e., data types) in **C**

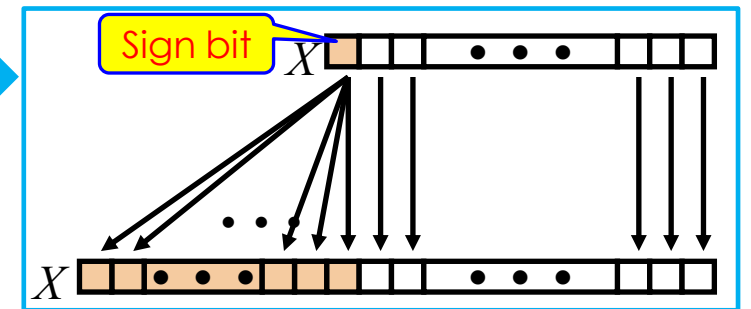
1. Small data type -> larger data type

➤ Sign extension (may require padding)

➤ Unsigned: zero extension



➤ Signed: sign bit extension



➤ Conclusion: Value unchanged

Converting unsigned (or signed) of different sizes (i.e., data types) in **C**

➤ Let's try:

➤ Going from a data type that has a width of 3 bits ($w = 3$) to a data type that has a width of 5 bits ($w = 5$)

➤ Unsigned: $x = 3 \Rightarrow 011_2 \quad w = 3$
new $x = \leq w = 5$

➤ Signed: $x = 3 \Rightarrow 011_2 \quad w = 3$ $x = -3 \Rightarrow 101_2 \quad w = 3$
new $x = \leq w = 5$ new $x = \leq w = 5$

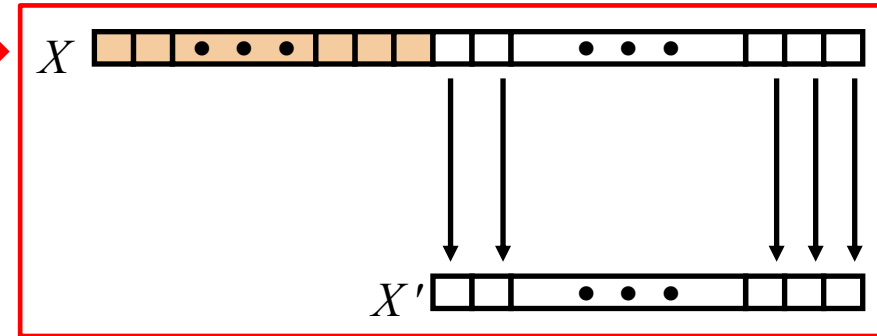
Converting unsigned (or signed) of different sizes (i.e., data types) in **C**

2. Large data type -> smaller data type

➔ Truncation



➔ Conclusion: Value may be altered



Converting unsigned (or signed) of different sizes (i.e., data types) in **C**

► Let's try:

- Going from a data type that has a width of 5 bits ($w = 5$) to a data type that has a width of 3 bits ($w = 3$)

► Unsigned: $x = 27 \Rightarrow 11011_2$ $w = 5$
new $x = \quad \leq \quad w = 3$

► Signed: $x = -15 \Rightarrow 10001_2$ $w = 5$
new $x = \quad \leq \quad w = 3$

$x = -1 \Rightarrow 11111_2$ $w = 5$
new $x = \quad \leq \quad w = 3$

Demo – Looking at *size* and *sign* conversions in **C**

- What does the demo illustrate?
 - Size conversion:
 - Converting to a larger (wider) data type -> Converting `short` to `int`
 - Converting to a smaller (narrower) data type -> Converting `short` to `char`
 - Sign conversion:
 - Converting from signed to unsigned -> Converting `short` to `unsigned short`
 - Converting from unsigned to signed -> Converting `unsigned short` to `short`
 - Size and Sign conversion:
 - Converting from signed to unsigned larger (wider) data type -> Converting `short` to `unsigned int`
 - Converting from signed to unsigned smaller (narrower) data type -> Converting `short` to `unsigned char`
- This demo (code and results) posted on our course web site

Lab 2 - Integral Representation and Impact of Fixed Size Memory in C

- In this lab, we shall play around with some C code, experiencing the impact of storing our data in memory which has a fixed size
- Posted on our course web site
- Nothing to hand in
- Worth 0 marks
- If you have questions?
 - Go to our lab periods on Monday and/or come to our office hours

Summary

reviewed in Lab 1

- Interpretation (encoding) of bit pattern **B** into either unsigned value **U** or signed value **T**
 - **B2U(X)** and **U2B(X)** encoding schemes (conversion)
 - **B2T(X)** and **T2B(X)** encoding schemes (conversion)
 - Signed value expressed as two's complement => **T**

- Conversions from unsigned <=> signed values

- Method 1: **U2T(X)** and **T2U(X)** => **U2B(X)** then **B2T(X)**, **T2B(X)** then **B2U(X)**
- Method 2: **U2T(X)** and **T2U(X)** => adding or subtracting 2^w

Seen in our demo today!

- Implication in **C**: when converting (implicitly via promotion and explicitly via casting):

- Sign:

- Unsigned <=> signed (of same size) -> **NOTE**: Both have same bit pattern, however, this bit pattern may be interpreted differently
 - Can have unexpected effects, i.e., producing a different value

- Size:

- Small -> large (for signed, e.g., **short** to **int** and for unsigned, e.g., **unsigned short** to **unsigned int**)
 - **sign extension**: For unsigned -> zeros extension, for signed -> sign bit extension
 - Both yield expected result -> resulting value is unchanged
- Large -> small (e.g., **unsigned int** to **unsigned short**)
 - **truncation**: Unsigned/signed -> most significant bits are truncated (discarded)
 - May not yield expected results -> original value may be altered

- Both (sign and size): 1) **size** conversion is executed first, then 2) **sign** conversion is executed
e.g., **int** to **unsigned short**: **int** to **short** then **short** interpreted as **unsigned short**

Next Lecture

- Storing data in memory

- “Under the Hood” - **von Neumann** architecture

- Bits and bytes in memory

Note that we do not know yet what data these bits represent!

- How to diagram memory (used in this course and other references)

- reviewed in Lab 0** ➤ How to express memory content -> **using binary and hexadecimal numeral system**

- Order of bytes in memory -> **endian**

- Bit manipulation – bitwise operations

- Boolean algebra + shifting

We shall start giving meaning to bits -> what data they represent.

- What kind of data do bits (in memory) represent -> **encoding scheme**

- Bits representing (encoding) integral numbers

Demo

- reviewed in Lab 1** ➤ Unsigned and signed

- practiced in Lab 2** ➤ Converting, expanding and truncating – Impact of fixed size memory

- Arithmetic operations

- Bits representing (encoding) real numbers

- IEEE floating point representation

- Floating point in C – casting, rounding, addition, ...